Atividade Avaliativa N1

Engenharia de Software | ADS 4º semestre | FAEX

Profa. Especialista Denise Ribeiro

Esta atividade integra os conteúdos das Semanas 6, 7 e 8. A equipe utilizará a modelagem já produzida para uma jornada do MVP, revisará a distribuição de responsabilidades e apresentará uma arquitetura inicial justificável. O trabalho será realizado integralmente durante a aula, das 19h às 22h, e deverá ser registrado no GitHub e no Ambiente Virtual até o horário final.

Informação	Regra
Data e horário	19h às 22h
Prazo final	22h, conforme registro do Ambiente Virtual
Valor	25 pontos
Equipe	2 a 4 integrantes
Escopo	Uma única jornada do MVP
Fonte oficial avaliada	Conteúdo existente na tag avaliacao-a2
Entrega no AVA	Individual, com links e respostas próprias em .**PDF** (não será aceito outro formato)

1 Situação profissional

A equipe já realizou o levantamento de requisitos e produziu modelos iniciais do MVP. Antes da implementação, é necessário verificar se a modelagem representa uma única jornada, se as responsabilidades estão organizadas e se existe uma estrutura técnica capaz de orientar o desenvolvimento. O resultado deve permitir que cliente e equipe técnica compreendam o que será construído e por que as principais decisões foram tomadas.

2 Objetivo e regra de escopo

Objetivo: transformar a modelagem existente de uma jornada do MVP em uma proposta inicial de projeto e arquitetura, com rastreabilidade, justificativas e evidências no GitHub.

Regra de escopo: a equipe trabalhará com somente uma jornada. O sistema inteiro não deverá ser modelado nesta atividade.

Regra de reaproveitamento: fluxograma, casos de uso, protótipo e classes da Semana 6 não serão refeitos do zero. Eles deverão ser reunidos, revisados e corrigidos apenas quando houver inconsistência.

3 Referencial teórico

3.1 Modelagem e rastreabilidade

Modelos representam perspectivas complementares do software. O fluxograma mostra como o processo acontece; o diagrama de casos de uso delimita atores e funcionalidades; o protótipo demonstra a interação; e o diagrama de classes representa conceitos, dados e relações. Esses artefatos devem descrever a mesma jornada. A rastreabilidade permite identificar qual requisito, critério ou regra justifica cada elemento.

3.2 Abstração e modularidade

Abstração significa representar somente características relevantes para a finalidade do modelo. Modularidade significa dividir o sistema em partes com responsabilidades compreensíveis. Uma divisão adequada facilita alteração, teste e manutenção.

3.3 Coesão e acoplamento

Coesão indica o quanto as responsabilidades de uma classe ou módulo pertencem ao mesmo propósito. Acoplamento indica o nível de dependência entre partes. Em geral, procura-se alta coesão e acoplamento controlado: cada módulo concentra uma responsabilidade e conhece somente as dependências necessárias.

3.4 Arquitetura inicial

A arquitetura inicial apresenta as partes principais do sistema, suas responsabilidades e relações. Nesta atividade, ela deve mostrar usuário, interface, aplicação, domínio, persistência e integrações externas realmente necessárias. Ela não é um desenho de infraestrutura em nuvem e não deve incluir tecnologias que a equipe ainda não escolheu.

3.5 Decisões técnicas

Uma decisão técnica profissional registra a situação, a escolha, a justificativa e a consequência. Não basta afirmar que uma tecnologia ou estrutura será usada; a equipe deve explicar por que ela atende ao MVP e quais limitações ou impactos produz.

4 Produtos obrigatórios

A entrega será composta por seis arquivos organizados exatamente como indicado:

```
docs/
└── avaliacao-a2/
    ├── README.md
    ├── modelagem/
    │   ├── modelagem-revisada.drawio
    │   └── modelagem-revisada.png
    ├── projeto/
    │   └── analise-de-projeto.md
    └── arquitetura/
        ├── arquitetura-inicial.drawio
        └── arquitetura-inicial.png
```

Arquivos fora dessa estrutura poderão ser desconsiderados quando não for possível identificar sua relação com a atividade.

5 Responsabilidade individual

Cada integrante deverá possuir uma responsabilidade principal, contribuir de forma verificável e entregar sua própria resposta no AVA. O produto é coletivo, mas a nota final é individual.

Papel	Responsabilidade principal
Requisitos e rastreabilidade	README, história, critérios, regra de negócio e rastreabilidade.
Modelagem	Revisão do fluxograma, casos de uso, protótipo e classes.
Projeto	Responsabilidades, módulos, coesão, acoplamento e melhoria.
Arquitetura e integração	Arquitetura inicial, decisões, pull request e integração.

Em equipes menores, uma pessoa poderá acumular papéis. A atribuição principal não dispensa o conhecimento da entrega completa.

6 Preparação do repositório

Horário sugerido: 19h15 às 19h30.

Contexto: alterações avaliativas não devem ser feitas diretamente na versão principal. A branch isola o trabalho e preserva o histórico.

6.1 Comandos

```
git status
git switch main
git pull origin main
git switch -c avaliacao-a2
git branch --show-current
```

Resultado esperado: o último comando deve apresentar avaliacao-a2.

Criar as pastas:

```
mkdir -p docs/avaliacao-a2/modelagem
mkdir -p docs/avaliacao-a2/projeto
mkdir -p docs/avaliacao-a2/arquitetura
```

6.2 Critério de aceite

☐ A branch atual é avaliacao-a2.

☐ A pasta docs/avaliacao-a2 foi criada.

☐ Nenhum arquivo da avaliação está sendo produzido diretamente na main.

6.3 Proibições

- Não apagar entregas anteriores.
- Não copiar arquivos de outra equipe.
- Não criar várias branches para a mesma avaliação.
- Não trabalhar diretamente na main.

7 Contexto e rastreabilidade

Horário sugerido: 19h30 às 19h50.

Contexto: todos os artefatos precisam representar a mesma jornada. O README registra o problema, a necessidade e a origem dos elementos modelados.

Criar docs/avaliacao-a2/README.md com o conteúdo abaixo:

```
# Avaliação A2 - Do modelo à arquitetura do MVP

## Equipe
| Integrante | Usuário GitHub | Responsabilidade | Evidência |
|---|---|---|---|
| | | | |

## Projeto
Nome:

## Problema
Descrever em até três linhas.

## Jornada selecionada
Informar apenas uma jornada.

## História de usuário
Como [papel], quero [objetivo], para [benefício].

## Critérios de aceitação
- CA-01:
- CA-02:
- CA-03:

## Regra de negócio
- RN-01:

## Rastreabilidade resumida
| Origem | Representação no projeto |
|---|---|
| História | |
| CA-01 | |
| CA-02 | |
| CA-03 | |
| RN-01 | |
```

Primeiro commit:

```
git add docs/avaliacao-a2/README.md
git commit -m "docs: define contexto da avaliacao A2"
```

7.1 Critério de aceite

☐ Existe somente uma jornada.

☐ A história possui papel, objetivo e benefício.

☐ Os três critérios são verificáveis.

☐ A regra de negócio interfere no comportamento.

☐ Todos os integrantes estão identificados com seus usuários do GitHub.

8 Modelagem revisada

Horário sugerido: 19h50 às 20h20.

Contexto: os modelos produzidos anteriormente são a entrada desta avaliação. Eles deverão ser reunidos no draw.io em quatro páginas: Fluxograma, Casos de uso, Protótipo e Classes.

1. Abrir o draw.io.
2. Criar um arquivo com quatro páginas nomeadas.
3. Copiar os modelos existentes para as páginas correspondentes.
4. Corrigir somente inconsistências relacionadas à jornada selecionada.
5. Salvar como modelagem-revisada.drawio.
6. Exportar uma imagem legível como modelagem-revisada.png.

Modelo	Verificação obrigatória
Fluxograma	A decisão corresponde à RN-01; início, ações, saídas e fim estão claros.
Casos de uso	Atores e funcionalidades correspondem à jornada e estão dentro da fronteira correta.
Protótipo	Cada botão corresponde a uma ação; toda informação exibida possui origem conhecida.
Classes	Classes são necessárias; atributos e multiplicidades refletem o negócio.

Segundo commit:

```
git add docs/avaliacao-a2/modelagem
git commit -m "docs: consolida modelagem revisada da jornada"
```

8.1 Critério de aceite

☐ O arquivo .drawio possui as quatro páginas.

☐ O PNG está legível.

☐ Os modelos representam a mesma jornada.

☐ Não foram incluídas funcionalidades sem requisito.

9 Análise de projeto

Horário sugerido: 20h35 às 21h00.

Contexto: a modelagem mostra o que o sistema realiza. A análise de projeto verifica como responsabilidades e dependências serão organizadas.

Criar docs/avaliacao-a2/projeto/analise-de-projeto.md:

```
# Análise de projeto

## Classes e responsabilidades
| Classe | Responsabilidade | O que não deve fazer |
|---|---|---|
| | | |

## Módulos
| Módulo | Responsabilidade | Classes ou funções |
|---|---|---|
| Interface | | |
| Aplicação | | |
| Domínio | | |
| Persistência | | |

## Coesão e acoplamento
| Elemento | Situação | Evidência |
|---|---|---|
| Coesão | Adequado/Atenção/Crítico | |
| Acoplamento | Adequado/Atenção/Crítico | |

## Melhoria
Problema encontrado:
Alteração realizada ou proposta:
Benefício esperado:

## Decisões técnicas
| Decisão | Escolha | Justificativa | Consequência |
|---|---|---|---|
| Organização da aplicação | | | |
| Persistência ou integração | | | |
```

Terceiro commit e primeira publicação:

```
git add docs/avaliacao-a2/projeto
git commit -m "docs: analisa responsabilidades e modulos"
git push -u origin avaliacao-a2
```

9.1 Critério de aceite

☐ Cada classe possui uma responsabilidade principal.

☐ Os quatro módulos foram analisados, mesmo quando algum não se aplica.

☐ Coesão e acoplamento possuem evidência, não apenas classificação.

☐ Uma melhoria foi identificada.

10 Arquitetura inicial

Horário sugerido: 21h00 às 21h30.

Contexto: a arquitetura transforma módulos e responsabilidades em uma visão técnica que orienta a implementação.

Criar arquitetura-inicial.drawio e arquitetura-inicial.png contendo:

- usuário ou ator;
- interface;
- aplicação;
- domínio ou regras de negócio;
- persistência;
- integração externa, somente quando existir;
- setas que indiquem comunicação ou dependência.

Regras: a interface pode depender da aplicação; a aplicação coordena a jornada; o domínio concentra regras; a persistência armazena e recupera informações. Não incluir AWS. Quando a tecnologia não estiver escolhida, usar o papel técnico, como Banco de dados.

Quarto commit:

```
git add docs/avaliacao-a2
git commit -m "docs: adiciona arquitetura inicial do MVP"
git push
```

10.1 Critério de aceite

☐ Todas as partes possuem nomes e responsabilidades reconhecíveis.

☐ As setas possuem direção.

☐ A arquitetura corresponde aos módulos descritos.

☐ As duas decisões possuem justificativa e consequência.

11 Pull request e revisão

Horário sugerido: 21h30 às 21h42.

7. No GitHub, selecionar Compare and pull request.
8. Confirmar base main e compare avaliacao-a2.
9. Usar o título A2 - Do modelo à arquitetura do MVP.
10. Cada integrante deverá registrar sua revisão individual.

```
## Revisão individual - [Nome]

Dimensão revisada: [Requisitos/Modelagem/Projeto/Arquitetura]
Item verificado:
Evidência encontrada:
Inconsistência identificada:
Conclusão: [Aprovado/Correção necessária]
```

Uma revisão escrita apenas como “está bom” não será considerada evidência.

12 Merge e tag

Horário sugerido: 21h42 às 21h55.

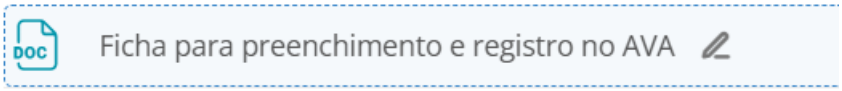
Após a revisão e eventual correção, realizar o merge da pull request. Depois executar:

```
git switch main
git pull origin main
git tag -a avaliacao-a2 -m "Entrega final da Avaliacao A2"
git push origin avaliacao-a2
git rev-parse --short HEAD
```

Resultado esperado: a tag avaliacao-a2 deve apontar para o commit final integrado à main.

13 Entrega individual no Ambiente Virtual

Horário reservado: 21h55 às 22h. O envio é individual e deve ser registrado até 22h.



Utilizar o modelo disponibilizado em word no ambiente virtual conforme imagem acima. O envio deve ser feito em PDF, arquivos word serão desconsiderados e nota inidvidual será zerada automaticamente.

Regra de prazo: será considerado o horário registrado no AVA. O conteúdo técnico será avaliado na tag avaliacao-a2. Alterações posteriores não modificarão a entrega avaliada.

Bom traballho à todos!

14 Solução de problemas previsíveis

Problema	Ação
A branch avaliacao-a2 já existe	Executar git switch avaliacao-a2.
O arquivo não apareceu no commit	Executar git status, depois git add docs/avaliacao-a2, commit e push.
O primeiro push falhou por falta de upstream	Executar git push -u origin avaliacao-a2.
Não consigo trocar de branch por alterações locais	Executar git status; adicionar e criar um commit antes de trocar. Não apagar alterações.
A pull request apresenta conflito	Registrar o conflito no AVA. A branch enviada antes das 22h poderá ser avaliada.
A tag já existe	Executar git tag e verificar se avaliacao-a2 já aponta para a entrega correta. Não recriar sem autorização.
mkdir -p não funciona no terminal utilizado	Criar manualmente as três pastas dentro de docs/avaliacao-a2.

15 Critérios de avaliação

A nota final da atividade será individual, composta por 15 pontos coletivos e 10 pontos individuais.

Critério coletivo	Pontos
Contexto, história, critérios e regra	2
Coerência da modelagem revisada	3
Classes, responsabilidades e modularidade	3
Coesão, acoplamento e melhoria	2
Arquitetura inicial	3
Decisões e rastreabilidade	2
Total coletivo	15

Critério individual	Pontos
Responsabilidade registrada	1
Contribuição relevante comprovada	3
Revisão técnica realizada	2

Critério individual	Pontos
Explicação de decisão	2
Compreensão da arquitetura e dependências	2
Total individual	10

16 Condições de correção

- Item completo, coerente e comprovado recebe pontuação integral.
- Item parcial recebe pontuação proporcional.
- Arquivo sem relação com a jornada pode receber no máximo 25% do critério.
- Item sem evidência na tag recebe zero.
- Quantidade de commits não substitui qualidade da contribuição.
- Nome registrado sem evidência não comprova participação.
- Respostas individuais idênticas não demonstram contribuição individual.
- Quem não entregar a parte individual no AVA perde os 10 pontos individuais.

17 Checklist final

☐ README completo.

☐ Quatro modelos reunidos no arquivo draw.io.

☐ Análise de projeto completa.

☐ Arquitetura editável e imagem legível.

☐ Duas decisões justificadas.

☐ Responsabilidade e evidência de cada integrante.

☐ Pull request aberta e revisada.

☐ Merge realizado.

☐ Tag avaliacao-a2 publicada.

☐ Todos enviaram individualmente no AVA até 22h.

Referências

FOWLER, Martin. UML distilled: a brief guide to the standard object modeling language. 3. ed. Boston: Addison-Wesley, 2004.

LARMAN, Craig. Applying UML and patterns. 3. ed. Upper Saddle River: Prentice Hall, 2004.

OBJECT MANAGEMENT GROUP. OMG Unified Modeling Language, version 2.5.1. Needham: OMG, 2017.

PRESSMAN, Roger S.; MAXIM, Bruce R. Software engineering: a practitioner's approach. 9. ed. New York: McGraw-Hill Education, 2021.

SOMMERVILLE, Ian. Software engineering. 10. ed. Harlow: Pearson, 2016.